ARTEMIS Prototype Development Plan

Technology stack, build sequence, and validation approach for the working prototype

Final Year Project | Department of Computer Science | FAST NUCES

Hira Khalid (23L-2594) | Doureesha Batool (23L-2651) | Hashir Rauf (23L-2572)

1. Purpose

This document lays out the technology stack and the sequence in which the prototype will be built, so the feature set in the main proposal can be validated for feasibility before the next semester begins. It is a companion to the ARTEMIS proposal and covers implementation only, not the research questions or evaluation design, which are defined separately.

2. Technology Stack

Core language and application layer

Python is the primary language for the entire prototype, covering the agent logic, the backend services, and the file, code, and document handling behind every feature. A single language keeps the codebase manageable for a three person team within a two semester timeline. The desktop interface will be built as a lightweight local application (a Python based desktop UI framework, such as PySide or a local web view served from the Python backend) so the interface, the workspace data, and the agent logic can run on the same machine without a separate deployment step.

Agentic AI layer

LangGraph is used to implement the agent that plans and executes multi step requests. It represents each user request as an explicit sequence of steps, with a defined pause point before any tool call that changes a file or reaches outside the workspace, such as sending an email or performing a web search. This maps directly onto the preview, approve, and undo pattern already defined in the proposal, and keeps the reasoning steps inspectable rather than hidden inside a single opaque prompt.

Language model access

Model access follows a local first order. A locally hosted open model, run through Ollama on the user's own machine, is used by default for requests that can be handled on device, keeping the system consistent with its bounded, privacy first design. If a request needs a model with stronger reasoning than the local model can reliably provide, or a local model is not available on the user's machine, the system falls back to a cloud model in this order: Gemini first, then OpenAI, then Anthropic. Each cloud provider is accessed through its standard API, and the fallback order is configurable so it can be adjusted after early testing shows which provider performs best for a given feature.

Storage

All workspace data, including tracked file metadata, saved notes, automation rules, and the activity log, is stored locally in SQLite. No workspace data is synced to a server by default. Document and code embeddings used for search and synthesis features are stored in a local vector index rather than a hosted vector database, to avoid a dependency that would not run fully offline.

Feature specific components

Feature	Key components
Pick Up Where You Left Off	File system watcher for change detection, local SQLite store
Tidy Up a Folder	Text extraction from documents, embedding based grouping
Pull Together Notes	Text extraction, citation formatting, local vector search
Notice a Repeated Task	Pattern matching over the logged action history
Code Assistant	Source file parsing, diff generation and review, undo log
Web Search	A search API call triggered only on an explicit user request

Mailing	Draft composer, an email provider API used only after approval
Document Drafting	Template based draft generation from workspace material
Speech Input and Output	Local speech to text and text to speech models where available, with a cloud fallback matching the language model fallback order

3. Context and Memory Architecture

ARTEMIS separates memory into three layers, each handled with a different mechanism rather than one general purpose memory system, since the features rely on different kinds of recall.

Working memory

The current request, the relevant part of the conversation, and any file or note directly involved are passed to the model as context for that call. This layer is kept deliberately small. Prompts are structured with a stable prefix, a fixed system description of ARTEMIS followed by a short workspace summary, so that repeated calls within the same session reuse as much of that prefix as possible. This is standard key value cache reuse at the model level, and it reduces the cost and latency of follow up requests without requiring ARTEMIS to manage the cache directly.

Long term and episodic memory

Session notes, the record of what changed since a workspace was last opened, automation rules, and the action and undo log are stored as structured rows in the local SQLite database already used for workspace data. These are handled as direct lookups, such as the five most recent files or the last three saved notes, rather than semantic search, since the questions asked of this layer are precise and small in volume.

Document and code knowledge

Pull Together Notes and the Code Assistant need to reason over material that does not fit in a single prompt. Files in a workspace are split into chunks, converted into embeddings, and stored in a local vector index scoped to that workspace. A request retrieves only the chunks most relevant to the question being asked, rather than sending whole documents to the model. Because a workspace holds a small, bounded set of files rather than a large open collection, a lightweight local index is sufficient and no hosted vector database is required. The index is rebuilt incrementally as files in the workspace change.

4. Build Sequence

The prototype is built in stages so that feasibility is proven early on the highest risk components, and each stage produces something that can be demonstrated before the next one begins.

Stage 1. Workspace foundation: Opt in workspace creation, the local SQLite store, the file system watcher, and the What ARTEMIS Knows panel. No AI model is required for this stage.

Stage 2. Agent and approval loop: The LangGraph agent, the local model connection with the cloud fallback chain, and the preview, approve, and undo interaction pattern, built once and reused by every later feature.

Stage 3. Core file features: Pick Up Where You Left Off, Tidy Up a Folder, and Notice a Repeated Task, since these depend only on the workspace foundation and the approval loop.

Stage 4. Document and code understanding: Pull Together Notes and the Code Assistant, which add text extraction, embeddings, and diff based review on top of the existing approval loop.

Stage 5. Outward facing actions: Web Search, Mailing, and Document Drafting, added after the approval loop has already been validated on lower risk, file only actions.

Stage 6. Speech input and output: Added last, as an alternate interaction mode layered on top of the already working text based flow for every other feature.

5. Feasibility Validation

Before the next semester begins, each stage above will be checked against three questions: does the local model handle the feature reliably enough on its own, how often does the system need to fall back to a cloud model, and does the approval loop add acceptable delay to a typical request. Stage 2 is treated as the highest priority risk to resolve first, since every later feature depends on it working correctly. Results from this validation will be used to

finalize which features are included in the first working version and which remain stretch goals, as already noted in the main proposal.

6. Open Questions To Resolve During Prototyping

- Which local model size is realistic to run on a typical student laptop without making the system feel slow.
- Which specific desktop UI approach gives the most reliable cross feature experience with the least setup effort.
- Whether local speech to text and text to speech models are accurate enough to avoid relying on a cloud fallback for most requests.
- How large a workspace's local vector index can grow before retrieval quality or rebuild time becomes noticeable, and whether that limit is realistic for a typical user.